

✓ 1. Quick Sort

- Tốt nhất:

Khi pivot chia mảng thành 2 phần đều nhau sau mỗi lần chia.

👉 Ví dụ: mảng [3, 1, 4, 2, 5], chọn pivot là 3 sẽ chia thành [1,2] và [4,5].

👉 Lúc này số bước chia ít, thời gian nhanh nhất $\rightarrow O(n \log n)$

- Tệ nhất:

Khi pivot luôn là giá trị lớn nhất hoặc nhỏ nhất $\rightarrow$ chia rất lệch.

👉 Ví dụ: mảng đã tăng dần [1, 2, 3, 4, 5], mà luôn chọn pivot đầu (1).

👉 Gọi đệ quy một phía $\rightarrow$ như sắp xếp nối tiếp $\rightarrow O(n^2)$

✓ 2. Bubble Sort

- Tốt nhất:

Mảng đã sắp xếp tăng dần sẵn như [1, 2, 3, 4, 5], và có kiểm tra "không hoán đổi" thì dừng sớm.

👉 Mỗi lần duyệt không đổi gì $\rightarrow$ kết thúc nhanh $\rightarrow O(n)$

- Tệ nhất:

Mảng ngược hoàn toàn như [5, 4, 3, 2, 1]

👉 Phải hoán đổi từng cặp $\rightarrow$ rất nhiều bước $\rightarrow O(n^2)$

✓ 3. Interchange Sort

- Tốt nhất & Tệ nhất:

Luôn so sánh mọi cặp phần tử, nên mảng thế nào cũng vậy.

👉 Không tối ưu được $\rightarrow$ luôn $O(n^2)$ dù đã sắp hay ngược.

✓ 4. Insertion Sort

- Tốt nhất:

Mảng đã sắp sẵn, ví dụ [1, 2, 3, 4, 5].

👉 Mỗi phần tử chỉ so sánh 1 lần, không cần dời $\rightarrow O(n)$

- Tệ nhất:
Mảng đảo ngược hoàn toàn như [5, 4, 3, 2, 1]
👉 Mỗi phần tử phải dời toàn bộ các phần tử trước → $O(n^2)$
-

✓ 5. Binary Insertion Sort

- Tốt nhất:
Mảng đã sắp xếp → tìm vị trí chèn rất nhanh bằng nhị phân, không cần dời.
👉 $O(n \log n)$
 - Tệ nhất:
Mảng đảo ngược → vẫn phải dời tất cả phần tử lớn hơn → dời như Insertion Sort
→ $O(n^2)$
-

✓ 6. Shell Sort

- Tốt nhất:
Mảng gần như sắp xếp và chọn khoảng gap tốt (Hibbard, Sedgewick).
👉 Giảm số lần dịch, giảm độ sâu → gần $O(n \log n)$
 - Tệ nhất:
Mảng ngược hoàn toàn và dùng gap không tốt (như $n/2, n/4, \dots$)
👉 Dãy phân cách không hiệu quả → chạy lâu hơn → $O(n^2)$
-

✓ 7. Selection Sort

- Tốt nhất & Tệ nhất:
Dù mảng đã sắp hay ngược, vẫn phải tìm phần tử nhỏ nhất mỗi vòng
👉 Không thể tối ưu được → luôn $O(n^2)$
-

✓ 8. Heap Sort

- Tốt nhất & Tệ nhất:
Luôn phải xây dựng heap và đẩy phần tử lên/xuống → luôn đều

👉 Không phụ thuộc mảng đầu vào $\rightarrow O(n \log_{10} n) O(n \log n) O(n \log n)$ mọi trường hợp

✅ 9. Natural Merge Sort

- **Tốt nhất:**
Mảng có nhiều dãy con tăng liên tiếp (runs) $\rightarrow$ ít merge hơn.
👉 Ví dụ: [1, 2, 3, 5, 4, 6, 7] có thể tách thành [1,2,3], [5], [4], [6,7]
👉 Merge nhanh $\rightarrow O(n) O(n) O(n)$
 - **Tệ nhất:**
Mảng hoàn toàn ngẫu nhiên, mỗi run chỉ dài 1 $\rightarrow$ phải merge nhiều $\rightarrow O(n \log_{10} n) O(n \log n) O(n \log n)$
-

✅ 10. K-way Merge Sort

- **Tốt nhất & Tệ nhất:**
Phụ thuộc số "k" (số mảng con).
👉 Dù dữ liệu thế nào, luôn merge $\log_k(n)$ tầng $\rightarrow O(n \log_{10} kn) O(n \log_k n) O(n \log kn)$
-

✅ 11. Counting Sort

- **Tốt nhất:**
Khi giá trị nhỏ, ví dụ mảng [1, 2, 3, 2, 1], max = 3.
👉 Mảng đếm nhỏ $\rightarrow$ xử lý nhanh $\rightarrow O(n+k) O(n+k) O(n+k)$
 - **Tệ nhất:**
Khi giá trị lớn như [1, 9999, 5000] $\rightarrow$ cần tạo mảng đếm dài đến 9999
👉 Tốn bộ nhớ, vẫn chạy nhưng không tối ưu $\rightarrow O(n+k) O(n+k) O(n+k)$
-

✅ 12. Radix Sort

- Tốt nhất:

Khi số có ít chữ số, ví dụ [123, 456, 789] → chỉ cần 3 vòng lặp.

👉 Mỗi chữ số sắp xếp bằng Counting Sort → rất nhanh → $O(d(n+k))O(d(n+k))O(d(n+k))$

- Tệ nhất:

Khi số có rất nhiều chữ số, ví dụ 64 chữ số → 64 lần chạy Counting Sort

👉 Mỗi lần vẫn chạy $O(n+k)O(n+k)O(n+k)$, nhưng nhân với số lần nhiều → chậm hơn chút.